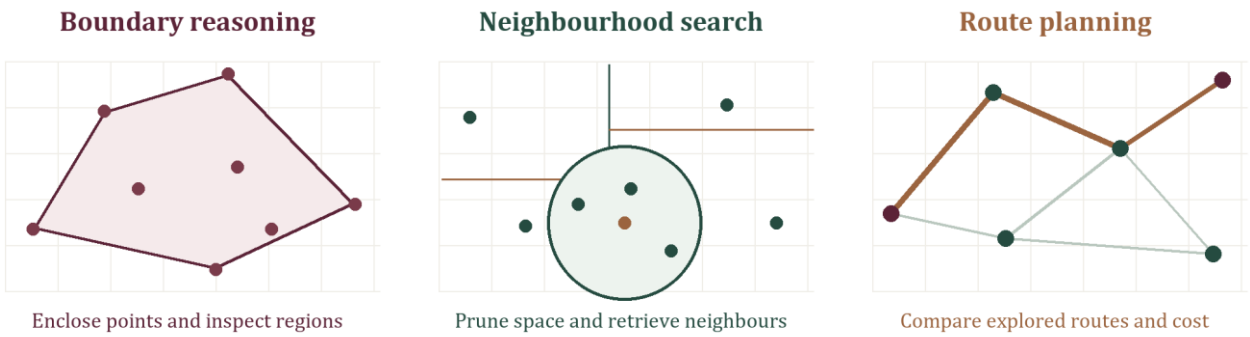
PROJECT PROPOSAL

Computational Geometry Visualizer

Satyam Dhal
Student ID b425050 Branch CSE-B
Course Object Oriented Programming
International Institute of Information Technology, Bhubaneswar

Project abstract

I propose to build an interactive C++ application using raylib to explain computational geometry and related algorithms through visual execution. Users will construct point sets, regions and weighted graphs, then inspect boundary construction, route exploration and density-based clustering. The prototype will combine convex hulls, geofence queries, Dijkstra, A star and DBSCAN in a shared workspace. Its outcome will be a reproducible visual laboratory for comparing algorithm decisions, with a clear extension towards security-event analysis.



Illustrative scenes for boundary construction, spatial lookup and graph routing.

Prototype scope and extensions

The five prototype modes cover convex hulls and polygon membership from computational geometry, Dijkstra and A star from graph search, and DBSCAN from density-based clustering. Shared orientation, distance and intersection operations support them. Inputs use bounded 2D scenes, non-negative graph weights and local sample datasets.

Closest-pair search and a k-d tree are optional spatial extensions. Indexed DBSCAN, geographic map integration and approved log ingestion belong to the larger Spatial Security Analytics Console. Keeping these extensions separate makes the initial deliverable specific while preserving a credible direction for growth.

Expected outcome

A working desktop visualizer will support scene editing, pause, single-step playback, trace rewind and parameter comparison. Each run will retain its input, settings and result summary. Success means correct reference results, readable explanations and repeatable execution. Operation counts will describe algorithm work independently of animation speed.

Boundaries and spatial queries

The spatial toolkit answers region and proximity questions. Hull construction and geofence queries belong to the prototype; closest-pair search and a k-d tree extend the same data model without enlarging the initial commitment.

Monotone chain convex hull

After sorting the points, the algorithm builds lower and upper chains using orientation tests. The animation will distinguish accepted vertices, candidates and removed points. Duplicate coordinates and collinear boundary points will follow a defined policy. The completed hull must be convex and contain every input point.

A hull can summarize the footprint of sensor readings or selected event locations. It can enclose empty space between groups, so the display will distinguish an enclosing boundary from a density cluster. Users can move a point and inspect how the hull changes.

Geofence queries and the shared kernel

Ray casting will classify a query location as inside, outside or on a simple polygon. Boundary tests run first, and an endpoint rule prevents double-counting vertices. The view will highlight the tested edges and ray crossings. This supplies an explainable permitted-region check for the security extension.

Orientation and segment intersection will validate polygon boundaries and support collision demonstrations. Distance calculations will share the same world-coordinate system. These operations form the geometry kernel rather than separate introductory modules.

Closest pair as a spatial extension

Divide and conquer solves two halves of a sorted point set, then checks a narrow strip around their dividing line. Keeping points in vertical order during merging preserves O(n log n) computation. The view will show recursive regions, candidate comparisons and the best distance. Duplicate coordinates produce zero distance.

This extension illustrates proximity screening in maps and sensor systems. It finds the closest pair anywhere in the dataset; a nearest-neighbour query instead asks which observation lies closest to a chosen location.

Spatial indexing extension

A balanced 2D k-d tree partitions points with alternating horizontal and vertical splits. Nearest search explores the nearer branch first, then visits another region only when it could improve the answer. Radius queries retrieve all points within a selected distance. The display will reveal visited and pruned regions.

This index can accelerate nearby-asset searches and supply DBSCAN neighbourhoods. Edits require rebuilding it. Queries will be checked against exhaustive search because performance depends on the dataset and a difficult query can still visit all points.

Search paths and discover groups

Dijkstra as the weighted routing baseline

Dijkstra repeatedly settles the least-cost frontier vertex and relaxes its outgoing edges. The graph will model junctions and non-negative edge costs, with distance as the main demonstration weight. Users can inspect tentative distances, predecessors and the final route. Blocking an edge creates a new scenario. This provides a transparent baseline for comparing A star on the same graph.

A star for map and obstacle navigation

A* (A star) ranks candidates by f(v) = g(v) + h(v), combining the cost already travelled with a lower-bound estimate to the goal. For distance-weighted map graphs, straight-line distance is valid when each edge costs at least the distance between its endpoints. A unit-cost four-neighbour grid uses Manhattan distance. The demonstration will compare route cost and expanded vertices with Dijkstra; the amount of exploration depends on the graph and heuristic.

These ideas are relevant to map navigation and robot motion planning; GPS supplies a position, while the graph search supplies a route. Time-based routing would require a compatible time heuristic, rather than reusing distance without conversion.

DBSCAN for density patterns and outlier review

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) uses a radius ε and minimum count MinPts. A core point has at least MinPts neighbours within ε, including itself. Border points neighbour a core point but lack that density; remaining points are noise. Cluster expansion follows connected core neighbourhoods. The animation will show the query circle and expansion queue, with controls for both parameters.

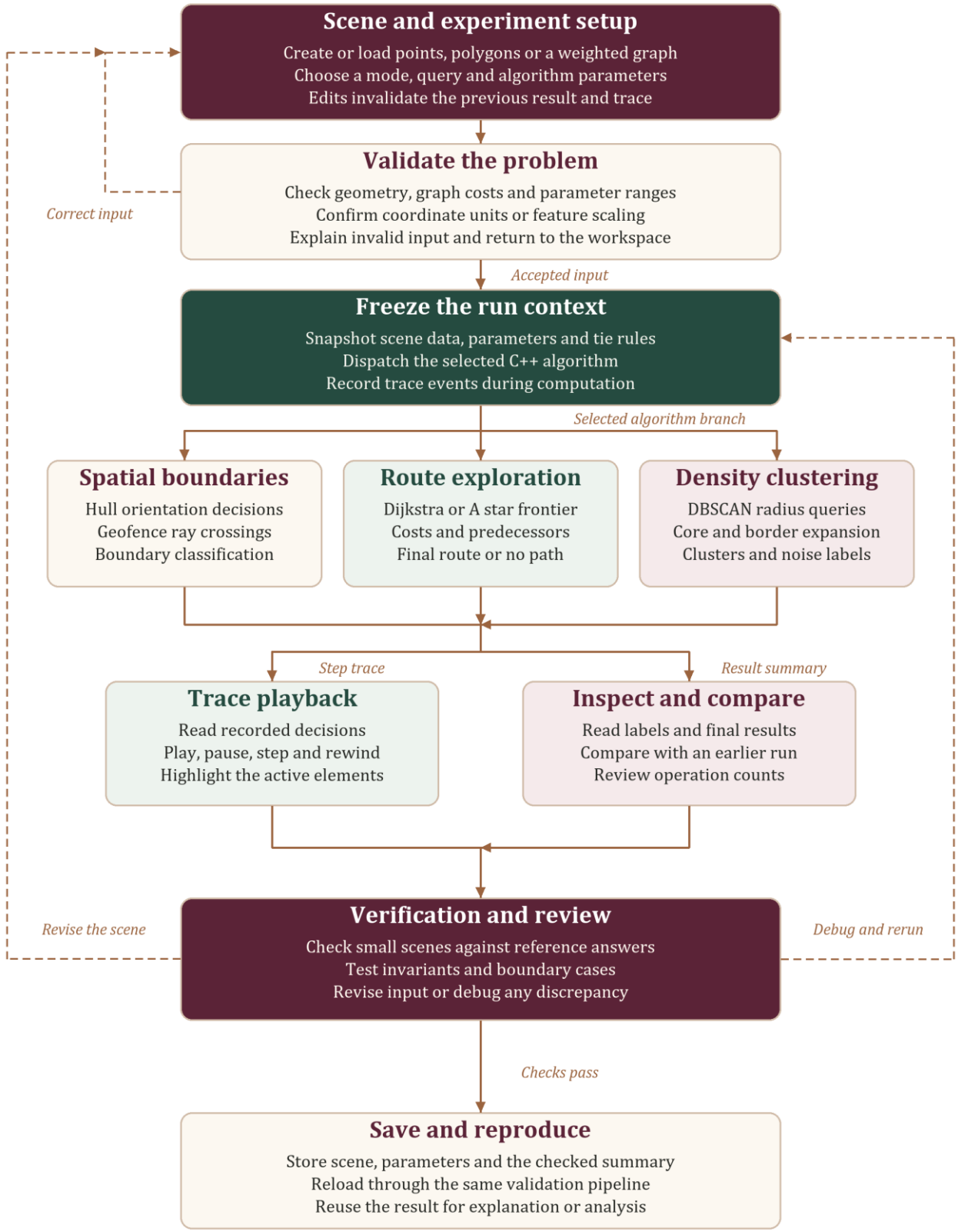
The prototype will use direct neighbourhood scans and a queue for cluster expansion. A fixed visit order makes ambiguous border assignments repeatable; a point initially marked as noise may later become a border point. A normalized security feature plane can reveal behaviour groups for review. Indexed acceleration is optional, and noise alone does not establish an attack.

Cost under the stated implementation

Module	Computational cost
Hull and optional closest pair	O(n log n) for each algorithm
Optional k-d tree	Expected O(n log n) build using median selection; O(n) worst-case query
Polygon membership	O(m) for one query against m polygon vertices
Dijkstra and A star	O((V + E) log V) with a binary heap and a consistent A star heuristic
DBSCAN baseline	$O(n^2)$ using direct scans; indexed performance depends on the data

n counts points; V and E count graph vertices and edges. Validation and trace storage are separate from these algorithm costs.

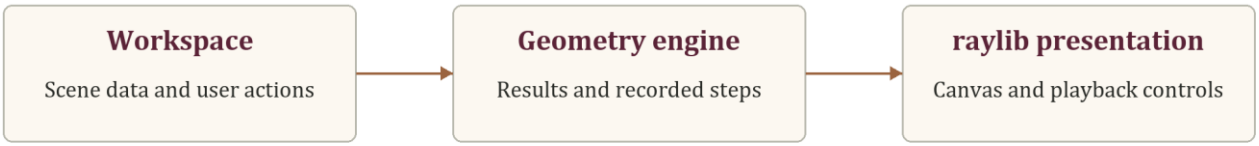
Execution and analytical review



Solid copper arrows carry execution and shared results. Dashed paths return a run for input correction or algorithm debugging.

C++ design with clear boundaries

All project logic will be written in C++. raylib will handle graphics and input; STL containers will support sorting, graph adjacency lists, queues and priority queues. Computation will produce data and trace events that can be inspected independently of rendering or playback timing.



Scene data enters the engine; recorded decisions and results feed the raylib presentation layer.

Responsibilities of the main components

Component	Responsibility
Scene and SceneIO	Store and reload points, polygons, graph edges, queries and parameters.
GeometryKernel	Provide orientation, distance, segment and boundary predicates.
SpatialIndex extension	Optionally build and query a k-d tree; rebuild after dataset changes.
AlgorithmRunner	Execute the five prototype modes through a shared run interface.
StepTrace and Playback	Record decisions and reconstruct states for pause, step and rewind.
CanvasRenderer	Draw geometry, partitions, frontiers, clusters and explanations in raylib.

An interface built for comparison

The canvas will support dragging and selection, with a compact control panel for algorithms, graph costs, query radius and DBSCAN parameters. Routing views will expose frontier scores and predecessors; clustering views will identify core, border and noise points. Colours will be reinforced with labels and line styles. Changes create a fresh run, while a saved summary permits comparison with a previous result.

Numerical discipline and reproducibility

The engine will work in double-precision world coordinates. Geometric tolerances will be documented, and sorting will retain a strict coordinate ordering. Spatial distances must use a suitable local projection for geographic data. Feature-space clustering will use documented scaling; changing units must not silently change the interpretation of a neighbourhood.

Every run will store a scene snapshot, parameters and deterministic tie rules. Trace events will preserve decisions and enough state for rewind. Scene and trace limits will bound memory use. Reloading repeats validation. The deliverable will include the C++ application, editable sample scenes and checked result summaries.

A target for spatial security analysis

Spatial Security Analytics Console

The larger target is an analyst console using authorized event records with pseudonymous identifiers linking each visual point to its source. It will offer two views: a geographic map for permitted-zone checks and proximity searches, and a scaled feature plane for behavioural patterns. User, device and time-window filters will define the population being examined.

Geofences can identify events outside approved regions; k-d tree queries can retrieve nearby observations. DBSCAN can group similar activity and expose isolated observations in a selected feature space. An analyst can inspect a point, its neighbours and the parameter choices that produced its label. Geography, cluster membership and noise will remain investigation context rather than automatic verdicts.

A concrete analysis scenario

For each device in a fixed observation window, plot request rate against failed-login rate after documented scaling. Run DBSCAN, inspect dense groups and noise, then open the underlying event summaries. In a separate map view, apply the permitted-region polygon. An evidence summary will preserve the window, filters, metrics and parameter values, making the analysis reproducible.

VPNs, travel, approximate IP geolocation and legitimate automated traffic can explain unexpected patterns. Evaluation will use labelled samples and analyst-reviewed false positives, with limited access to event data. The prototype will begin with local sample files; a later C++ adapter can connect an approved log source.

Other real world problems

Problem	Algorithms and intended use
Maps and logistics	Dijkstra and A star compare routes through weighted road graphs.
Robotics and sensors	Grid search avoids obstacles; nearest and radius queries inspect surroundings.
GIS and incident analysis	Geofences, hulls and DBSCAN examine region membership and spatial groups.

Acceptance criteria

Dijkstra and A star must agree on optimal route cost under the stated heuristic rules. Unreachable routes must be reported clearly. Hulls must pass convexity and containment checks; geofences must handle edges and vertices explicitly. DBSCAN must match core, border and noise classifications on unambiguous hand-labelled scenes. Changing inputs must invalidate the old trace, and reloading a saved scene must reproduce its result. Optional proximity modules will be checked against exhaustive search.

Technical references

raylib API · CGAL spatial searching · Boost graph search · Original DBSCAN paper